

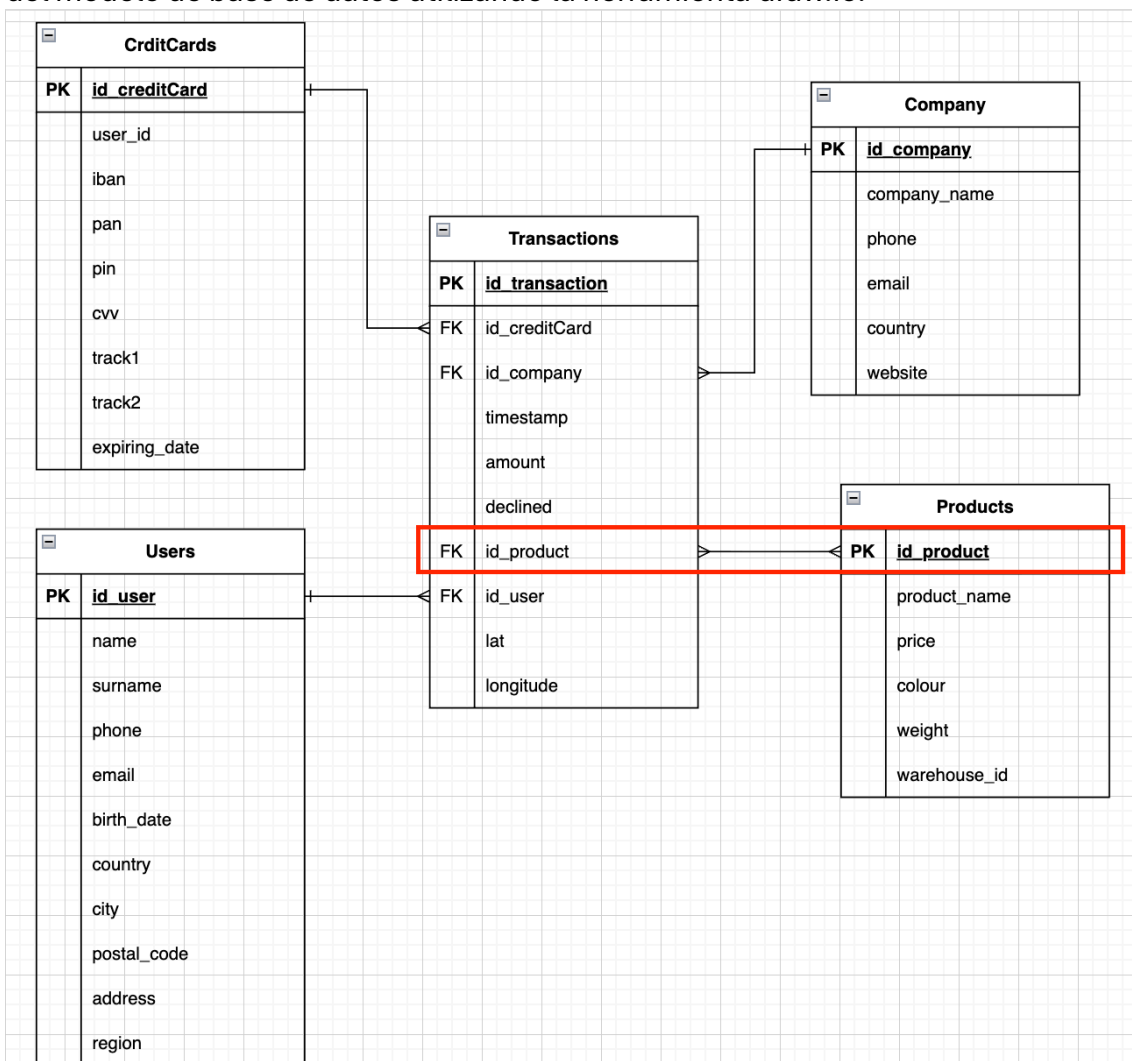
Nivell 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes.

A partir de los 6 archivos CSV proporcionados:

- american_users.csv
- companies.csv
- credit_cards.csv
- european_users.csv
- products.csv
- transactions.csv

Se realiza un análisis exploratorio de los datos y, posteriormente, un primer diseño del modelo de base de datos utilizando la herramienta draw.io.



En este diseño se plantea la unificación de los archivos de usuarios americanos y europeos en una única tabla users, con el objetivo de centralizar la información.

El modelo se estructura siguiendo un modelo en estrella, donde la tabla transactions actúa como fact table. Esta tabla se relaciona a su vez con cuatro tablas de dimensiones: users, companies, credit card y products.

Las dimensiones se relacionan con la tabla de hechos mediante relaciones de uno a muchos, donde las claves primarias de las dimensiones se corresponden con claves foráneas en la tabla Transactions.

En el caso de la tabla products, se identifica una relación de muchos a muchos, ya que una transacción puede incluir varios productos y, a su vez, un mismo producto puede aparecer en múltiples transacciones. Por este motivo, esta relación se abordará más adelante mediante una tabla intermedia que permita descomponer correctamente los productos asociados a cada transacción y facilite su análisis individual.

Para llevar este esquema al entorno de base de datos, se crea inicialmente la base de datos en MySQL junto con las dos tablas de usuarios originales, users_eua_raw y users_ue_raw, con el objetivo de cargar los datos y analizar posteriormente cómo unificarlos en una sola tabla. En esta primera fase, ambas tablas se definen admitiendo valores nulos y con los campos en formato VARCHAR(255), de manera que la manipulación de los datos resulte más flexible.

Además, se incorpora un campo adicional denominado region, que no está presente en los archivos CSV originales, pero que permitirá identificar el origen de cada registro como americano o europeo. Este campo resulta útil para unificar posteriormente ambas tablas en una única dimensión users, sin perder la información sobre la procedencia geográfica de los usuarios.

```
4 -- User EUA
5 CREATE TABLE users_eua_raw (
6     id_user VARCHAR(255) NULL,
7     name VARCHAR(255) NULL,
8     surname VARCHAR(255) NULL,
9     phone VARCHAR(255) NULL,
10    email VARCHAR(255) NULL,
11    birth_date VARCHAR(255) NULL,
12    country VARCHAR(255) NULL,
13    city VARCHAR(255) NULL,
14    postal_code VARCHAR(255) NULL,
15    address VARCHAR(255) NULL,
16    region VARCHAR(255) NULL
17 );
18
19 -- User UE
20 CREATE TABLE users_ue_raw (
21     id_user VARCHAR(255) NULL,
22     name VARCHAR(255) NULL,
23     surname VARCHAR(255) NULL,
24     phone VARCHAR(255) NULL,
25     email VARCHAR(255) NULL,
26     birth_date VARCHAR(255) NULL,
27     country VARCHAR(255) NULL,
28     city VARCHAR(255) NULL,
29     postal_code VARCHAR(255) NULL,
30     address VARCHAR(255) NULL,
31     region VARCHAR(255) NULL
32 );
```

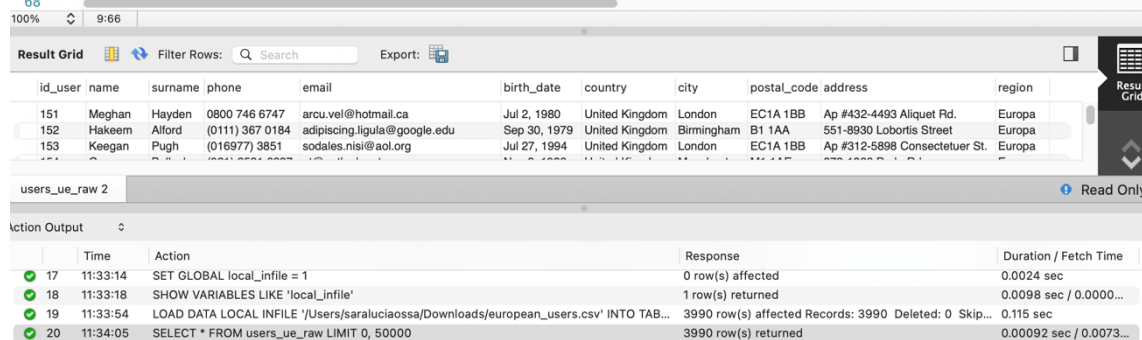
	Time	Action	Response	Duration / Fetch Time
16	11:29:17	CREATE TABLE users_ue_raw (id_user VARCHAR(255) NULL, name VARCHAR(255) NULL,...	0 row(s) affected	0.0090 sec

Creadas las tablas provisionales de users, y para poder ejecutar la importación de los datos del CSV, en mac, se ejecuta este comando en la terminal:

```
/usr/local/mysql/bin/mysql --local-infile=1 -h localhost -P 3306 -u root -p
```

Luego se importan los primeros datos de users europeos y se verifica que se hayan cargado correctamente en cada uno de los campos, como se muestra en la siguiente imagen:

```
51 -- NIVEL 0
52 -- Preparar la carga (antes activar comando en terminal)
53 • SET GLOBAL local_infile = 1;
54 • SHOW VARIABLES LIKE 'local_infile';
55
56 -- Estructura para cargar datos user europeos
57 • LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/european_users.csv'
58 INTO TABLE users_ue_raw
59 FIELDS TERMINATED BY ','
60 ENCLOSED BY '"'
61 LINES TERMINATED BY '\n'
62 IGNORE 1 ROWS
63 (id_user, name, surname, phone, email, birth_date, country, city, postal_code, address, region)
64 SET region = 'Europa';
65
66 • SELECT *
67 FROM users_ue_raw;
```



	id_user	name	surname	phone	email	birth_date	country	city	postal_code	address	region
151	Meghan	Hayden	0800 746 6747	arcu.vel@hotmail.ca	Jul 2, 1980	United Kingdom	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.	Europa	
152	Hakeem	Alford	(0111) 367 0184	adipiscing.ligula@google.edu	Sep 30, 1979	United Kingdom	Birmingham	B1 1AA	551-8930 Lobortis Street	Europa	
153	Keegan	Pugh	(016977) 3851	sodales.nisi@aol.org	Jul 27, 1994	United Kingdom	London	EC1A 1BB	Ap #312-5898 Consectetur St.	Europa	

	Time	Action	Response	Duration / Fetch Time
✓ 17	11:33:14	SET GLOBAL local_infile = 1	0 row(s) affected	0.0024 sec
✓ 18	11:33:18	SHOW VARIABLES LIKE 'local_infile'	1 row(s) returned	0.0098 sec / 0.0000...
✓ 19	11:33:54	LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/european_users.csv' INTO TAB...	3990 row(s) affected Records: 3990 Deleted: 0 Skip...	0.115 sec
✓ 20	11:34:05	SELECT * FROM users_ue_raw LIMIT 0, 50000	3990 row(s) returned	0.00092 sec / 0.0073...

Nota: se utiliza SET dentro de LOAD DATA para asignar automáticamente el valor “Europa” a la columna region en todas las filas importadas.

Este paso se repite para cargar la tabla de users americanos, verificando que se haya cargado la región de America:

```

69 -- Estructura para cargar datos user americanos
70 • LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/american_users.csv'
71 INTO TABLE users_eua_raw
72 FIELDS TERMINATED BY ','
73 ENCLOSED BY '"'
74 LINES TERMINATED BY '\n'
75 IGNORE 1 ROWS
76 (id_user, name, surname, phone, email, birth_date, country, city, postal_code, address)
77 SET region = 'America';
78
79 • SELECT *
80 FROM users_eua_raw;
81
82

```

id_user	name	surname	phone	email	birth_date	country	city	postal_code	address	region
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	United States	New York	10001	348-7818 Sagittis St.	America
2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	Aug 23, 1992	United States	Philadelphia	19101	903 Sit Ave	America
3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	Apr 29, 1998	United States	Houston	77001	736-2063 Tellus St.	America

users_eua_raw 3

Time	Action	Response	Duration / Fetch Time
19 11:33:54	LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/european_users.csv' INTO TAB...	3990 row(s) affected Records: 3990 Deleted: 0 Skip...	0.115 sec
20 11:34:05	SELECT * FROM users_ue_raw LIMIT 0, 50000	3990 row(s) returned	0.00092 sec / 0.0073...
21 11:36:40	LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/american_users.csv' INTO TAB...	1010 row(s) affected Records: 1010 Deleted: 0 Skipp...	0.023 sec
22 11:36:47	SELECT * FROM users_eua_raw LIMIT 0, 50000	1010 row(s) returned	0.00085 sec / 0.000...

Nota: al igual que en la tabla anterior, se utiliza SET dentro de LOAD DATA para asignar automáticamente el valor “America” a la columna region en todas las filas importadas.

Seguidamente, se procede a revisar si hay duplicados o nulos en los id_user de las tablas users_eua_raw y users_ue_raw. Esto se hace con el objetivo de evitar duplicidad para unir las posteriormente en una sola tabla de users.

```

82 -- Contar si hay duplicados o nulos en los id_user
83 • WITH users AS (
84     SELECT id_user FROM users_eua_raw
85     UNION ALL
86     SELECT id_user FROM users_ue_raw
87 )
88 SELECT id_user, COUNT(*)
89 FROM users
90 GROUP BY id_user
91 HAVING COUNT(*) > 1;
92
93 • SELECT id_user
94 FROM users
95 WHERE id_user IS NULL
96     OR TRIM(id_user) = ''; -- TRIM para eliminar espacios vacíos en caso de que venga más de uno
97
98

```

id_user
NULL

Result 4 users 5

Time	Action	Response	Duration / Fetch Time
24 11:39:08	CREATE TABLE users (id_user VARCHAR(50) NOT NULL, name VARCHAR(255) NULL,...	0 row(s) affected	0.011 sec
25 11:41:15	WITH users AS (SELECT id_user FROM users_eua_raw UNION ALL SELECT id_user...	0 row(s) returned	0.011 sec / 0.000008...
26 11:41:15	SELECT id_user FROM users WHERE id_user IS NULL OR TRIM(id_user) = '' LIMIT 0, 50000	0 row(s) returned	0.0019 sec / 0.00000...

Nota: se utiliza UNION ALL para mantener los duplicados y poder detectar si algún id_user está repetido entre ambas tablas. Posteriormente, se agrupan los resultados por id_user y se filtran aquellos que aparecen más de una vez. Asimismo,

también se utiliza TRIM para eliminar espacios en blanco al inicio y al final de cada fila de la columna id_user, permitiendo detectar como válidos registros cuyo id_user no es NULL pero pueda contener espacios en blanco.

Tras verificar que no existen valores duplicados ni nulos en el campo id_user, se procede a la creación de la tabla users, definiendo este campo como PK y NOT NULL. Dado que los datos han sido previamente validados, se ajusta también la longitud de este campo a VARCHAR(50).

La tabla incluye todos los atributos de las tablas de users america y europa e incorpora, además, el campo region, que permitirá diferenciar el origen de los usuarios tras la unificación de los datos.

```
34 -- Users
35 • CREATE TABLE users (
36     id_user VARCHAR(50) NOT NULL,
37     name VARCHAR(255) NULL,
38     surname VARCHAR(255) NULL,
39     phone VARCHAR(255) NULL,
40     email VARCHAR(255) NULL,
41     birth_date VARCHAR(255) NULL,
42     country VARCHAR(255) NULL,
43     city VARCHAR(255) NULL,
44     postal_code VARCHAR(255) NULL,
45     address VARCHAR(255) NULL,
46     region VARCHAR(255) NULL,
47     PRIMARY KEY (id_user)
48 );
49
50
51
100% 3:48
Action Output
Time Action Response Duration / Fetch Time
24 11:39:08 CREATE TABLE users ( id_user VARCHAR(50) NOT NULL, name VARCHAR(255) NULL,... 0 row(s) affected 0.011 sec
```

A continuación, y una vez asegurada la unicidad de los campos de id_userd, se realiza la unificación de las tablas users_eua_raw y users_ue_raw mediante UNION ALL, insertando los registros en la tabla final users.

Finalmente, se verifica que todos los registros se hayan cargado correctamente en la tabla resultante.

```

98 -- Unir las dos tablas de users (eua y ue) en la nueva tabla "Users"
99 • INSERT INTO users (id_user, name, surname, phone, email, birth_date, country, city, postal_code, address, region)
100 SELECT id_user, name, surname, phone, email, birth_date, country, city, postal_code, address, region
101 FROM users_eua_raw
102 UNION ALL
103 SELECT id_user, name, surname, phone, email, birth_date, country, city, postal_code, address, region
104 FROM users_ue_raw;
105
106 • SELECT *
107 FROM users
108 ORDER BY id_user DESC;

```

id_user	name	surname	phone	email	birth_date	country	city	postal_code	address	region
999	Jxnziq	Swketlik	+70-735-4881	jxnziq.swketlik@example.com	Jan 26, 1995	Portugal	Lisbon	1000-001	513 Swketlik St	Europa
998	Senxyp	Wjliptg	+76-168-2201	senxyp.wjliptg@example.com	Jan 11, 1950	Canada	Edmonton	T5A 0A1	179 Wjliptg St	America
997	Uxhpfr	Nqljwbek	+89-348-5083	uxhpfr.nqljwbek@example.com	Apr 24, 1998	Netherlands	Utrecht	3511	712 Nqljwbek St	Europa
996	Fnjbqu	Kepprupi	+36-736-3647	mjbqu.kepprupi@example.com	Mar 22, 1970	Canada	Calgary	T1Y 0A1	973 Kepprupi St	America
995	Dtwhih	Sekkhlas	+44-164-2511	dtwhih.sskkhlask@example.com	Sep 3, 1962	Netherlands	Utrecht	3511	146 Sekkhlask St	Europa
994	Iniktn	Fcmtrcm	+31-575-4832	iniktn.fcmtrcm@example.com	Mar 14, 1970	Italy	Milan	20100	543 Fcmtrcm St	Europa

Nota: una vez comprobado que no existen duplicados en id_user, se utiliza UNION ALL en el INSERT INTO para unificar ambas tablas de forma más eficiente, evitando la comprobación y eliminación de duplicados que realiza UNION.

Al comprobar que los datos hayan sido correctamente cargados, y entre las dos tablas haya 5000 registros, se eliminan las tablas de users americanos y europeos que sirvieron para dar origen a la tabla de users.

```

110 -- Se eliminan las tablas de EUA y UE
111 • DROP TABLE IF EXISTS users_eua_raw, users_ue_raw;
112
113 •

```

Time	Action	Response	Duration / Fetch Time
11:53:14	DROP TABLE IF EXISTS users_eua_raw, users_ue_raw;	0 row(s) affected, 1 warning(s): 1051 Unknown table 'world_transactions.users_eu_raw'	0.0089 sec
11:53:32	DROP TABLE IF EXISTS users_eua_raw, users_ue_raw;	0 row(s) affected, 1 warning(s): 1051 Unknown table 'world_transactions.users_eua_raw'	0.0082 sec

Teniendo la tabla de users ya cargada, se crea la tabla de company, credit_card y transactions:

```

60 CREATE TABLE credit_card (
61     id_creditCard VARCHAR(255) NULL,
62     id_user VARCHAR(255) NULL,
63     iban VARCHAR(255) NULL,
64     pan VARCHAR(255) NULL,
65     pin VARCHAR(255) NULL,
66     cvv VARCHAR(255) NULL,
67     track1 VARCHAR(255) NULL,
68     track2 VARCHAR(255) NULL,
69     expiring_date VARCHAR(255) NULL
70 );
71
72 -- Transactions (fact table)
73 CREATE TABLE transactions (
74     id_transaction VARCHAR(255) NULL,
75     id_creditCard VARCHAR(255) NULL,
76     id_company VARCHAR(255) NULL,
77     timestamp VARCHAR(255) NULL,
78     amount VARCHAR(255) NULL,
79     declined VARCHAR(255) NULL,
80     id_product VARCHAR(255) NULL,
81     id_user VARCHAR(255) NULL,
82     lat VARCHAR(255) NULL,
83     longitude VARCHAR(255) NULL
84 );

```

	Time	Action	Response	Duration / Fetch Time
30	20:26:06	CREATE TABLE company (id_company VARCHAR(255)...	0 row(s) affected	0.0097 sec
31	20:26:12	CREATE TABLE credit_card (id_creditCard VARCHAR(255)...	0 row(s) affected	0.0083 sec
32	20:26:17	CREATE TABLE transactions (id_transaction VARCHAR(255)...	0 row(s) affected	0.011 sec

A continuación, se cargan los datos de la tabla company. Una vez realizada la carga, se ejecuta una consulta de verificación para comprobar que los datos se han importado correctamente y que cada campo corresponde con su columna definida en el CREATE TABLE.

```

53 -- Cargar datos de company
54 LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/companies.csv'
55 INTO TABLE company
56 FIELDS TERMINATED BY ','
57 ENCLOSED BY '"'
58 LINES TERMINATED BY '\n'
59 IGNORE 1 ROWS
60 (id_company, company_name, phone, email, country, website);
61
62 SELECT *
63 FROM company;
64

```

id_company	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars

	Time	Action	Response	Duration / Fetch Time
38	20:34:40	CREATE TABLE company (id_company VARCHAR(255)...	0 row(s) affected	0.0091 sec
39	20:34:49	LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Download...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.0052 sec
40	20:34:56	SELECT * FROM company LIMIT 0, 50000	100 row(s) returned	0.00083 sec / 0.000...

Seguidamente, se realiza la carga de la tabla credit card siguiendo el mismo procedimiento. Posteriormente, se verifica la correcta importación de los datos mediante una consulta SELECT *, asegurando la consistencia de los campos y su correcta asignación a la tabla.

```

65 -- Cargar datos de Credit Card
66 • LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/credit_cards.csv'
67 INTO TABLE credit_card
68 FIELDS TERMINATED BY ','
69 ENCLOSED BY '"'
70 LINES TERMINATED BY '\n'
71 IGNORE 1 ROWS
72 (id_creditCard, id_user, iban, pan, pin, cvv, track1, track2, expiring_date);
73
74 • SELECT *
75 FROM credit_card;

```

id_creditCa...	id_user	iban	pan	pin	cvv	track1	track2	exp
CcU-2938	275	TR301950312213576817638661	5424465566813633	3257	984	%B8383712448554646*WovsxkjDpwiev*86041...	%B7653863056044187=800716333673	10/
CcU-2945	274	DO26854763748537475216568689	5142423821948828	9080	887	%B4621311609958661*UftuyfsSeimxn*061062...	%B4149568437843501=510714033071	08/
CcU-2952	273	BG45IVQL52710525608255	4556 453 55 5287	4598	438	%B2183285104307501*CddytcUxwfdq*59079...	%B6778580257827162=6906859740077	06/

Time	Action	Response	Duration / Fetch Time
40 20:34:56	SELECT * FROM company LIMIT 0, 50000	100 row(s) returned	0.00083 sec / 0.000...
41 20:35:52	LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Download...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.059 sec
42 20:36:07	SELECT * FROM credit_card LIMIT 0, 50000	5000 row(s) returned	0.00095 sec / 0.0078...

Por último, se realiza la carga de la tabla Transactions. Para esta importación fue necesario ajustar el delimitador de campos a “;” ya que el archivo CSV no utilizaba el separador “,” que traía los otros documentos. Una vez completado el proceso, se verifica la correcta importación de los datos mediante una consulta exploratoria sobre la tabla.

```

77 -- Cargar datos de transactions
78 • LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/transactions.csv'
79 INTO TABLE transactions
80 FIELDS TERMINATED BY ';'
81 LINES TERMINATED BY '\n'
82 IGNORE 1 ROWS
83 (id_transaction, id_creditCard, id_company, timestamp, amount, declined, id_product, id_user, lat, longitude);
84
85 • SELECT *
86 FROM transactions;

```

id_transaction	id_creditCa...	id_company	timestamp	amount	declined	id_product	id_user	lat	longitude
CDDA7E40-544D-47BB-A4ED-671DD8A950D9	CcS-6894	b-2466	2018-12-12 08:05:17	161.88	0	75, 73, 98	2313	59.62050974356148	16.559977155728436
09456357-8E9B-475A-8257-87A023699964	CcS-5135	b-2342	2024-05-20 22:49:08	171.13	0	3	554	45.76458841901318	4.843056518287656
C47C7C84-C174-4973-A76B-825A9DB5582A	CcS-8415	b-2250	2018-11-04 23:16:18	497.29	0	92, 85, 36, 23	3834	52.06849608409912	4.301099382555438

Time	Action	Response	Duration / Fetch Time
47 20:45:04	CREATE...	0 row(s) affected	0.0088 sec
48 20:45:14	LOAD D...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	0.461 sec
49 20:45:20	SELECT...	50000 row(s) returned	0.00061 sec / 0.046...

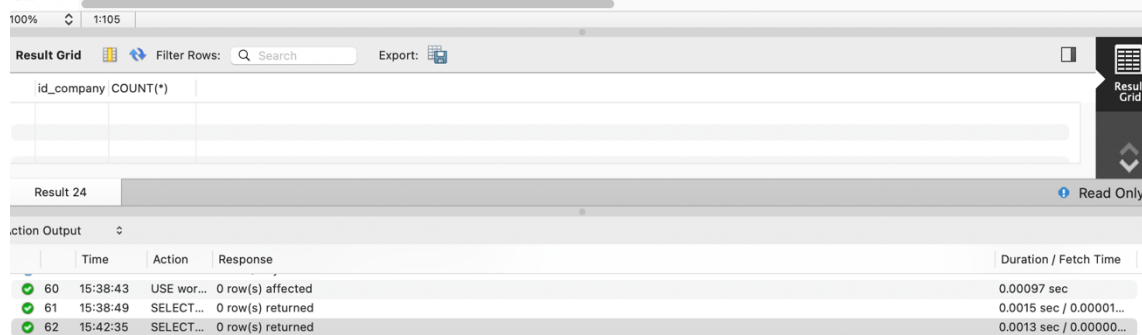
Nota: la columna id_product se carga inicialmente como un campo que puede contener varios identificadores en una misma fila (separados por comas), tal y como vienen en el archivo. Se mantiene de forma provisional para facilitar la carga inicial de los datos. Posteriormente, esta estructura se transformará mediante la creación de una tabla intermedia que permita representar correctamente la relación entre transacciones y productos.

Una vez cargadas todas las tablas (excepto products, que su relación requiere un tratamiento específico), se procede a la fase de limpieza y preparación del modelo.

En esta etapa se revisan los datos y se definen las PK y FK, además de ajustar los tipos de datos de algunas columnas previamente cargadas como VARCHAR.

En primer lugar, se descarta la existencia de valores duplicados en los campos que actuarán como claves primarias (id_company, id_creditCard e id_transaction), con el objetivo de garantizar la unicidad antes de definir las restricciones.

```
90 -- Revisar si hay duplicados en los id (futuras PK) de cada de las tablas transactions, credit_card y company
91 • SELECT id_company, COUNT(*)
92 FROM company
93 GROUP BY id_company
94 HAVING COUNT(id_company) > 1;
95
96 • SELECT id_creditCard, COUNT(*)
97 FROM credit_card
98 GROUP BY id_creditCard
99 HAVING COUNT(id_creditCard) > 1;
100
101 • SELECT id_transaction, COUNT(*)
102 FROM transactions
103 GROUP BY id_transaction
104 HAVING COUNT(id_transaction) > 1;
105
106
107
```



The screenshot shows a database management tool interface. At the top, there's a SQL editor with three queries to check for duplicates in the 'id' fields of 'company', 'credit_card', and 'transactions' tables. Below the editor, the 'Result Grid' shows the results of the first query, which is empty. The 'Action Output' pane at the bottom shows the execution of three queries, all returning 0 rows, indicating no duplicates were found.

	Time	Action	Response	Duration / Fetch Time
60	15:38:43	USE wor...	0 row(s) affected	0.00097 sec
61	15:38:49	SELECT...	0 row(s) returned	0.0015 sec / 0.00001...
62	15:42:35	SELECT...	0 row(s) returned	0.0013 sec / 0.00000...

A continuación, se comprueba si estos mismos campos contienen valores nulos o valores vacíos, asegurando así su validez para ser definidos como claves primarias.

```

106 -- Revisar si hay valores nulos en los id (futuras PK) de cada de las tablas transactions, credit_card y company
107 • SELECT *
108 FROM company
109 WHERE id_company IS NULL
110 OR TRIM(id_company) = ' '; -- TRIM para eliminar espacios vacíos en caso de que venga más de uno
111
112 • SELECT *
113 FROM credit_card
114 WHERE id_creditCard IS NULL
115 OR TRIM(id_creditCard) = ' ';
116
117 • SELECT *
118 FROM transactions
119 WHERE id_transaction IS NULL
120 OR TRIM(id_transaction) = ' ';
121
122
123

```

Nota: siguiendo el mismo criterio aplicado en las tablas anteriores, se utiliza TRIM para eliminar espacios en blanco al inicio y al final de los campos, permitiendo identificar correctamente valores vacíos o no válidos.

```

129 -- Modificar campos y añadir las PK de las tablas transactions, credit_card y company
130 • ALTER TABLE company
131     MODIFY id_company VARCHAR(10) NOT NULL,
132     MODIFY company_name VARCHAR(255) NOT NULL,
133     ADD CONSTRAINT pk_company PRIMARY KEY (id_company);
134
135 • ALTER TABLE credit_card
136     MODIFY id_creditCard VARCHAR(10) NOT NULL,
137     MODIFY id_user VARCHAR(50) NULL,
138     ADD CONSTRAINT pk_credit_card PRIMARY KEY (id_creditCard);
139
140 • ALTER TABLE transactions
141     MODIFY id_transaction VARCHAR(255) NOT NULL,
142     MODIFY id_user VARCHAR(50) NULL,
143     MODIFY id_company VARCHAR(10) NULL,
144     MODIFY id_creditCard VARCHAR(10) NULL,
145     MODIFY amount DECIMAL(10,2) NULL,
146     ADD CONSTRAINT pk_transactions PRIMARY KEY (id_transaction);
147
148 • ALTER TABLE transactions
149     MODIFY declined BOOLEAN NULL;
150
151

```

100%

1:151

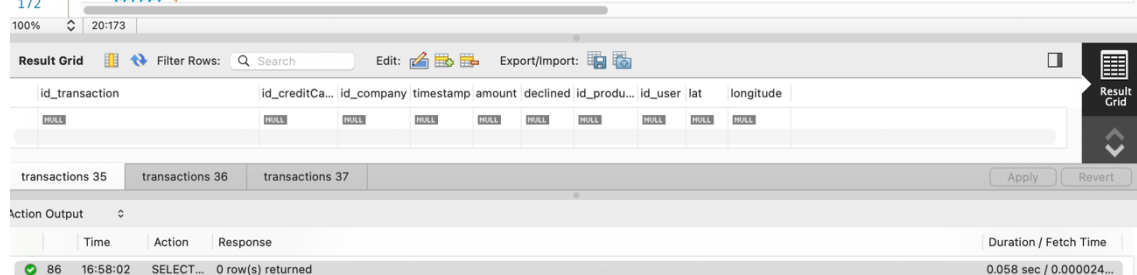
Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 75	16:22:04	ALTER T...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.055 sec
⚠ 76	16:22:07	ALTER T...	100000 row(s) affected, 1 warning(s): 1681 Integer display width is deprecated and will be removed in a future release. Records: 100000...	0.457 sec
✗ 77	16:23:38	ALTER T...	Error Code: 1068. Multiple primary key defined	0.0013 sec
✓ 78	16:24:31	ALTER T...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.0065 sec

Nota: Los campos de tipo ID se ajustan a un tamaño de VARCHAR más adecuado para optimizar su uso. Asimismo, el campo declined se modifica a tipo booleano para representar correctamente valores binarios (0-1) de los datos de origen.

Seguidamente, y antes de añadir las claves foráneas (FK) en la tabla transactions, se realiza una validación de los datos de esta tabla (que servirá como fact table), para comprobar que los ID en ella, tienen correspondencia con los de las tablas de dimensiones.

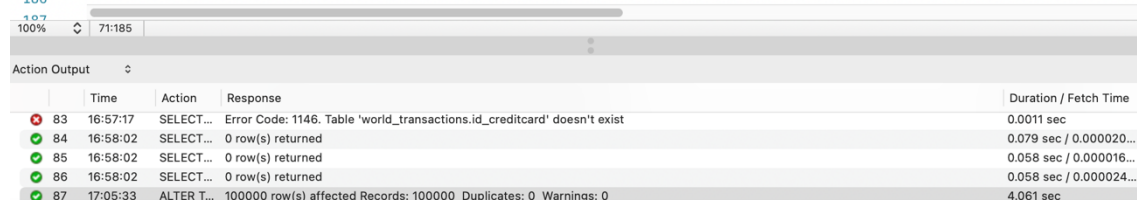
```
152 -- Validación fact table con dimensiones (antes de añadir FK a transactions)
153 • SELECT *
154 FROM transactions AS t
155 WHERE NOT EXISTS (
156     SELECT 1
157     FROM company AS c
158     WHERE c.id_company = t.id_company
159 );
160
161 • SELECT *
162 FROM transactions AS t
163 WHERE NOT EXISTS (
164     SELECT 1
165     FROM credit_card AS cc
166     WHERE cc.id_creditCard = t.id_creditCard
167 );
168
169 • SELECT *
170 FROM transactions AS t
171 WHERE NOT EXISTS (
172     SELECT 1
```



The screenshot shows a SQL IDE interface. The top pane displays three SQL queries for validation. The bottom pane shows the 'Result Grid' with columns: id_transaction, id_creditCa..., id_company, timestamp, amount, declined, id_produ..., id_user, lat, longitude. Below the grid, there are tabs for 'transactions 35', 'transactions 36', and 'transactions 37'. The 'Action Output' pane at the bottom shows a log of actions, including a successful SELECT query at 16:58:02 that returned 0 rows.

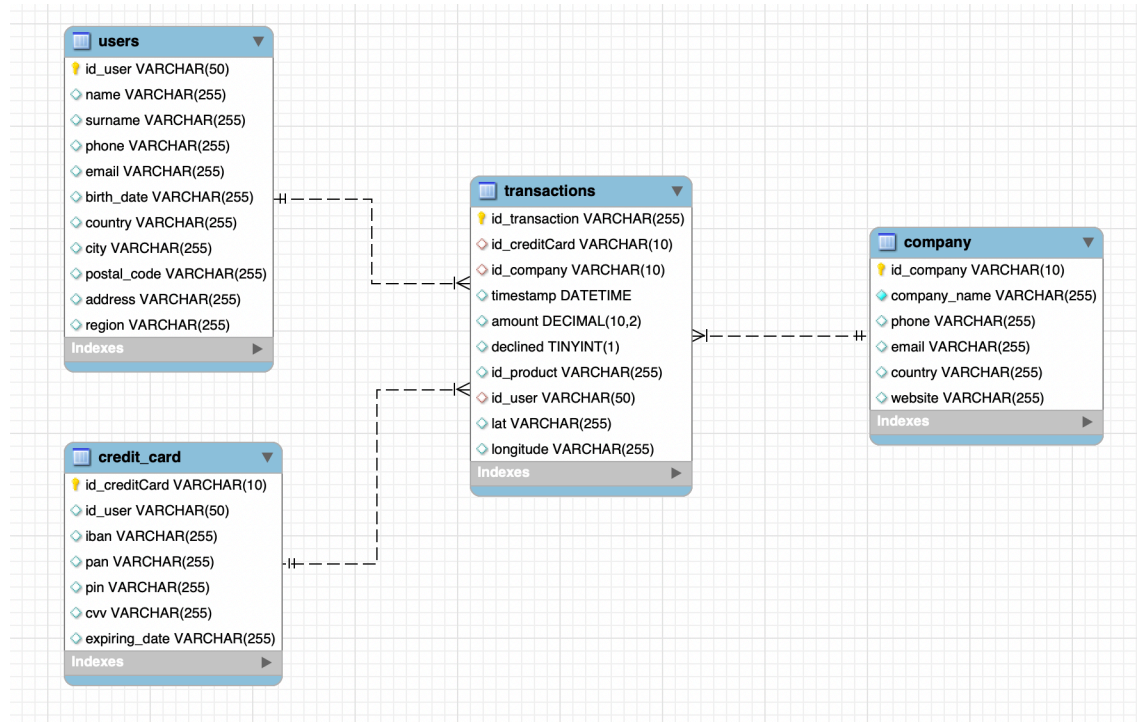
Tras validar que no existen registros huérfanos en las relaciones, se procede a crear las claves foráneas (FK) en la tabla transactions:

```
178 -- Añadir las FK a la factTable (transactions)
179 • ALTER TABLE transactions
180 ADD CONSTRAINT fk_transactions_user
181 FOREIGN KEY (id_user) REFERENCES users(id_user),
182 ADD CONSTRAINT fk_transactions_company
183 FOREIGN KEY (id_company) REFERENCES company(id_company),
184 ADD CONSTRAINT fk_transactions_creditcard
185 FOREIGN KEY (id_creditCard) REFERENCES credit_card(id_creditCard);
186
```



The screenshot shows the same SQL IDE interface. The top pane displays the SQL query to add foreign key constraints to the transactions table. The bottom pane shows the 'Action Output' log. It includes an error at 16:57:17 (Error Code: 1146, Table 'world_transactions.id_creditcard' doesn't exist) and three successful SELECT queries at 16:58:02 that returned 0 rows. The final action at 17:05:33 is an ALTER TABLE statement that affected 100,000 rows with 0 duplicates and 0 warnings.

Finalmente, se verifica que las tablas estén correctamente relacionadas mediante sus claves primarias (PK) y claves foráneas (FK), tal y como se refleja en el siguiente modelo:



Con las PK y las FK ya creadas, se procede a pulir más aun las tablas para mejorar el modelado:

Primero se convierten las columnas de tipo fecha desde VARCHAR a un tipo de dato adecuado en SQL, comenzando por la columna timestamp de la tabla transactions:

```

201 • DESCRIBE company;
202 • DESCRIBE credit_card;
203 • DESCRIBE transactions;
204 • DESCRIBE users;
205
206 -- Arreglar timestamp en transactions
207 -- Verificar si hay formato nulo o vacíos
208 • SELECT timestamp
209 FROM transactions
210 WHERE timestamp IS NULL
211 OR timestamp = '';
212
213 -- Se modifica la columna timestamp de transactions en formato DATETIME
214 • ALTER TABLE transactions
215 MODIFY COLUMN timestamp DATETIME;
216

```

Field	Type	Null	Key	Default	Extra
id_transaction	varchar(255)	NO	PRI		
id_creditCard	varchar(10)	YES	MUL		
id_company	varchar(10)	YES	MUL		
timestamp	datetime	YES			
amount	decimal(10,2)	YES			
declined	tinyint(1)	YES			
id_product	varchar(255)	YES			
id_user	varchar(50)	YES			

Result 51

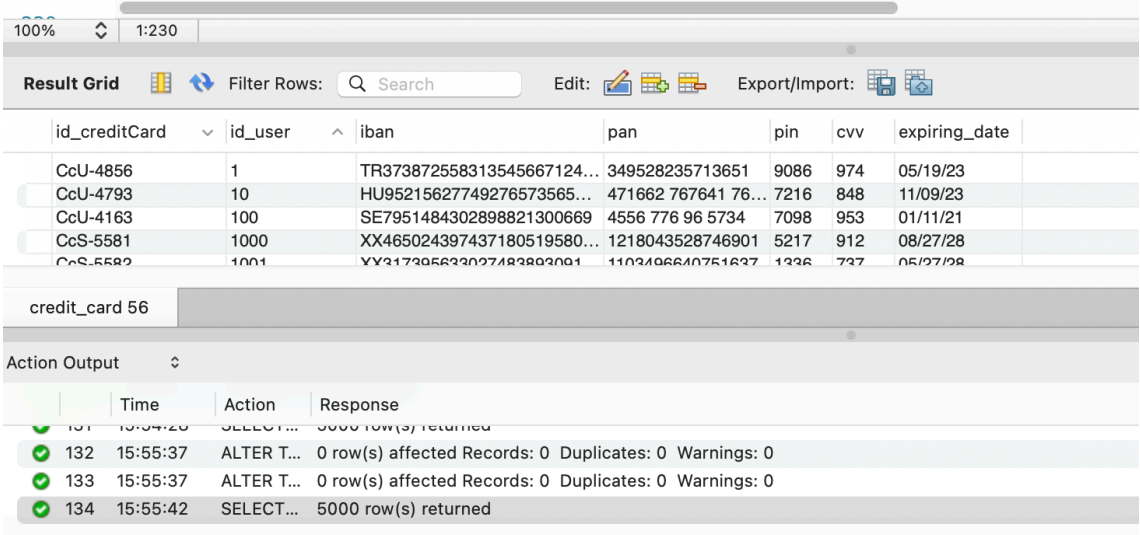
Action Output

	Time	Action	Response	Duration / Fetch Time
121	15:17:53	ALTER T...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	4.043 sec
122	15:18:24	DESCRI...	10 row(s) returned	0.0026 sec / 0.00001...

Nota: Se utiliza el tipo DATETIME en lugar de TIMESTAMP porque este último tiene un rango limitado (1970–2038) y aplica conversiones automáticas de zona horaria (UTC), lo que podría alterar los valores originales. En este caso, DATETIME permite preservar los datos tal como están en el dataset, lo que se considera más adecuado para el análisis de estos datos.

Se eliminan los campos track1 y track2 de la tabla credit_card que no aportan información relevante para la tabla. Dejando la tabla de credit card más limpia:

```
219
220 -- Se eliminan los campos track 1 y track 2 de credit_card
221 • ALTER TABLE credit_card
222 DROP track1;
223
224 • ALTER TABLE credit_card
225 DROP track2;
226
227 • SELECT *
228 FROM credit_card;
```



The screenshot shows a database management tool interface. The top section displays the SQL commands being executed. Below the commands, the 'Result Grid' shows the structure of the 'credit_card' table, including columns: id_creditCard, id_user, iban, pan, pin, cvv, and expiring_date. The 'Action Output' pane at the bottom shows the execution results of the commands, indicating that the columns were successfully dropped and the table was queried.

id_creditCard	id_user	iban	pan	pin	cvv	expiring_date
CcU-4856	1	TR373872558313545667124...	349528235713651	9086	974	05/19/23
CcU-4793	10	HU95215627749276573565...	471662 767641 76...	7216	848	11/09/23
CcU-4163	100	SE7951484302898821300669	4556 776 96 5734	7098	953	01/11/21
CcS-5581	1000	XX465024397437180519580...	1218043528746901	5217	912	08/27/28
CcS-5582	1001	YY317395632027483893091	1103496640751637	1336	737	05/27/28

credit_card 56

Time	Action	Response
15:55:28	SELECT...	5000 row(s) returned
15:55:37	ALTER T...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
15:55:37	ALTER T...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
15:55:42	SELECT...	5000 row(s) returned

Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```

1 • USE world_transactions;
2
3 -- NIVEL 1
4 -- Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
5 • SELECT *
6   FROM users AS u
7  WHERE EXISTS (
8      SELECT 1
9      FROM transactions AS t
10     WHERE t.id_user = u.id_user
11     GROUP BY t.id_user
12     HAVING COUNT(t.id_user) > 80
13 );
14

```

Result Grid

id_user	name	surname	phone	email	birth_date	country	city	postal_code	address
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany	Stuttgart	70173	82 Hwcru Street
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa	16100	53 Astuw Street
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	Aug 28, 1962	Poland	Gdansk	80-001	52 Xgvfridxs Street

users 3

Action Output

	Time	Action	Response	Duration / Fetch Time
88	17:19:36	SELECT...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right s...	0.00063 sec
89	17:19:50	USE wor...	0 row(s) affected	0.00071 sec
90	17:19:55	SELECT...	4 row(s) returned	0.041 sec / 0.000011...
91	17:20:05	SELECT...	4 row(s) returned	0.068 sec / 0.00000...
92	17:20:31	SELECT...	4 row(s) returned	0.067 sec / 0.000009...

Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```

15 -- Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.
16 • SELECT
17     cc.iban,
18     ROUND(AVG(t.amount), 2) AS transaccion_media
19   FROM transactions AS t
20  JOIN credit_card AS cc
21    ON cc.id_creditCard = t.id_creditCard
22  JOIN company AS c
23    ON c.id_company = t.id_company
24  WHERE c.company_name = 'Donec Ltd'
25  GROUP BY cc.iban;
26
27

```

Result Grid

iban	transaccion_media
XX911406401125586307596805	356.25
SK9446370242474562577506	142.96
XX776752917845952975555840	257.37
XX413827362289719304908990	139.59
XX347787246070769610780308	240.41
XX688768436543090894854602	188.58
MK28368851538688349	439.39
PL76249283566852676343404576	541.56

Result 30

Action Output

	Time	Action	Response	Duration / Fetch Time
182	18:15:32	SELECT	cc.iban, ROUND(AVG(t.am...	0.011 sec / 0.000056...

Nivell 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

```

33 • CREATE TABLE card_status AS
34     WITH tres_ultimas AS (
35         SELECT id_creditCard, declined, timestamp,
36             ROW_NUMBER() OVER (
37                 PARTITION BY id_creditCard
38                 ORDER BY timestamp DESC
39             ) AS ranking_tres
40         FROM transactions
41     )
42     SELECT id_creditCard,
43         CASE
44             WHEN SUM(declined) = 3 THEN 'inactiva'
45             ELSE 'activa'
46         END AS estado_tarjeta
47     FROM tres_ultimas
48     WHERE ranking_tres <= 3
49     GROUP BY id_creditCard;
50
51 • SELECT *
52     FROM card_status;

```

100% 25:49 1 error found

Result Grid Filter Rows: Search Export:

id_creditCa...	estado_tarje...
CcS-4857	activa
CcS-4858	activa

card_status 24 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
152	17:17:31	SELECT...	5000 row(s) returned	0.0014 sec / 0.0022 s...

Exercici 1

Quantes targetes estan actives?

```

57 • SELECT COUNT(*) AS tarjetas_activas
58     FROM card_status
59     WHERE estado_tarjeta = 'activa';
60
61

```

100% 33:59

Result Grid Filter Rows: Search Export:

tarjetas_activas
4995

Result 26 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
153	17:18:30	SELECT...	1 row(s) returned	0.0078 sec / 0.00000...
154	17:18:47	SELECT...	1 row(s) returned	0.0080 sec / 0.00001...

Nivell 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Para realizar este procedimiento, se crea la tabla products, que había quedado pendiente en el modelado inicial y se cargan los datos en ella:

```
86 -- Crear la tabla productos
87 • CREATE TABLE products (
88     id_product VARCHAR(255) NULL,
89     product_name VARCHAR(255) NULL,
90     price VARCHAR(255) NULL,
91     colour VARCHAR(255) NULL,
92     weight VARCHAR(255) NULL,
93     warehouse_id VARCHAR(255) NULL
94 );
```

100% 3:94

Action Output

	Time	Action	Response	Duration / Fetch Time
156	17:33:51	CREATE TABLE transac...	Error Code: 1050. Table 'transactions' already exists	0.00082 sec
157	17:33:57	CREATE TABLE produc...	0 row(s) affected	0.012 sec

Carga de datos en tabla products:

```
231 -- Estructura para cargar datos user europeos
232 • LOAD DATA LOCAL INFILE '/Users/saraluciaossa/Downloads/products.csv'
233 INTO TABLE products
234 FIELDS TERMINATED BY ','
235 ENCLOSED BY '"'
236 LINES TERMINATED BY '\n'
237 IGNORE 1 ROWS
238 (id_product, product_name, price, colour, weight, warehouse_id);
239
240 • SELECT *
241 FROM products;
```

100% 15:241

Result Grid

id_produ...	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	\$161.11	#7c7c7c	1	WH-4
2	Tarly Stark	\$9.24	#919191	2	WH-3
3	duel tourney Lannister	\$171.13	#d8d8d8	1.5	WH-2
4	warden south duel	\$71.89	#111111	3	WH-1
5	chancellor punk	\$171.22	#d8d8d8	2.2	WH-1

products 58

Action Output

	Time	Action	Response	Duration / Fetch Time
156	17:33:51	CREATE TABLE transac...	Error Code: 1050. Table 'transactions' already exists	0.00082 sec
157	17:33:57	CREATE TABLE produc...	0 row(s) affected	0.012 sec
158	17:37:57	LOAD DATA LOCAL INF...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.0048 sec

Seguidamente se procede a revisar los campos ID de la tabla products para revisar si hay nulos o duplicados. Al comprobarse que no los hay, se procede a añadir la PK:

```

244 -- Revisar si hay duplicados en los ID de products
245 • SELECT id_product, COUNT(*)
246 FROM products
247 GROUP BY id_product
248 HAVING COUNT(id_product) > 1;
249
250 -- Revisar si hay valores nulos en los id (futuras PK) de la tabla products
251 • SELECT *
252 FROM products
253 WHERE id_product IS NULL
254 OR TRIM(id_product) = ' ';
255
256 -- Añadir PK a products
257 • ALTER TABLE products
258 MODIFY id_product VARCHAR(255) NOT NULL,
259 ADD CONSTRAINT pk_product PRIMARY KEY (id_product);
260
261
262

```

id_produ...	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	\$161.11	#7c7c7c	1	WH-4
10	Karstark Dorne	\$119.52	#f4f4f4	2.4	WH-5
100	snuth duel	\$40.43	#6d6d6d	3	WH-95

products 61

Time	Action	Response	Duration / Fetch Time
167 17:48:26	SELECT * FROM produ...	100 row(s) returned	0.00058 sec / 0.000...

En la tabla transactions (fact table), la columna id_product contiene varios productos en una sola celda (73, 50, 34), separados por comas. Como se había mencionado inicialmente, esto implica una relación de tipo muchos a muchos entre transacciones y productos, que no está correctamente relacionada para hacer consultas en SQL.

Para solucionar esta relación, se crea una tabla intermedia que permita asociar cada transacción con sus productos de forma adecuada. Esta tabla facilitará el análisis y las consultas sobre los datos:

```

66 -- Se crea tabla intermedia entre transactions y products
67 • CREATE TABLE transactions_products (
68     id_transaction VARCHAR(50),
69     id_product VARCHAR(255),
70     PRIMARY KEY (id_transaction, id_product)
71 );
72
73 • INSERT INTO transactions_products (id_transaction, id_product)
74

```

Time	Action	Response	Duration / Fetch Time
173 17:57:19	CREATE TABLE transactions_products (...)	0 row(s) affected	0.0094 sec

Nota: se usa una clave primaria compuesta (id_transaction, id_product) para asegurarse que un mismo producto no se repita dentro de la misma transacción.

Posteriormente, se transforma la columna id_product utilizando JSON_TABLE, separando la lista de productos y convirtiéndola en múltiples filas, de modo que cada producto quede representado de forma individual.

```

INSERT INTO transactions_products (id_transaction, id_product)
74 SELECT
75     t.id_transaction,
76     jt.id_product
77 FROM transactions AS t
78 CROSS JOIN JSON_TABLE( -- Por cada fila de transactions, genera varias filas (una por cada producto)
79     CONCAT( -- Se concatenan los nuevos replace con la estructura del array del JSON '[' y ']'
80         '[',
81         REPLACE(REPLACE(t.id_product, ' ', ''), ',', ', '),
82         ']'
83     ),
84     '$[*]' COLUMNS ( -- recorre el JSON ($) y todos los elementos del array (*) y añadelo a una columna
85         id_product VARCHAR(255) PATH '$' -- PATH $ de cada elemento coge el valor completo y añadelo a una fila de la columna
86     )
87 ) AS jt;
88
89
90 • SELECT *
91 FROM transactions_products;
92
100% 28:91

```

Result Grid

id_transaction	id_product
00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
00043A49-2949-494B-A5DD-A5BAE3BB19DD	87

transactions_products 2

Action Output

	Time	Action	Response	Duration / Fetch Time
9	15:34:05	SELECT * FROM transactions_products LIMIT 0, 50000	50000 row(s) returned	0.00093 sec / 0.024...

Nota: La creación de esta tabla podría realizarse de forma más sencilla y eficiente utilizando Pandas en Python, mediante funciones como `.str.split()` y `.explode()` para separar los valores y generar la tabla intermedia. Sin embargo, dado que el ejercicio se plantea íntegramente en SQL, se ha optado por resolverlo utilizando `JSON_TABLE`.

Una vez creada la tabla intermedia `transactions_products` con los valores correspondientes, se definen las claves foráneas (FK) para relacionarla con las tablas `Transactions` y `Products`.

De este modo, se establece correctamente la relación de muchos a muchos entre ambas tablas.

```

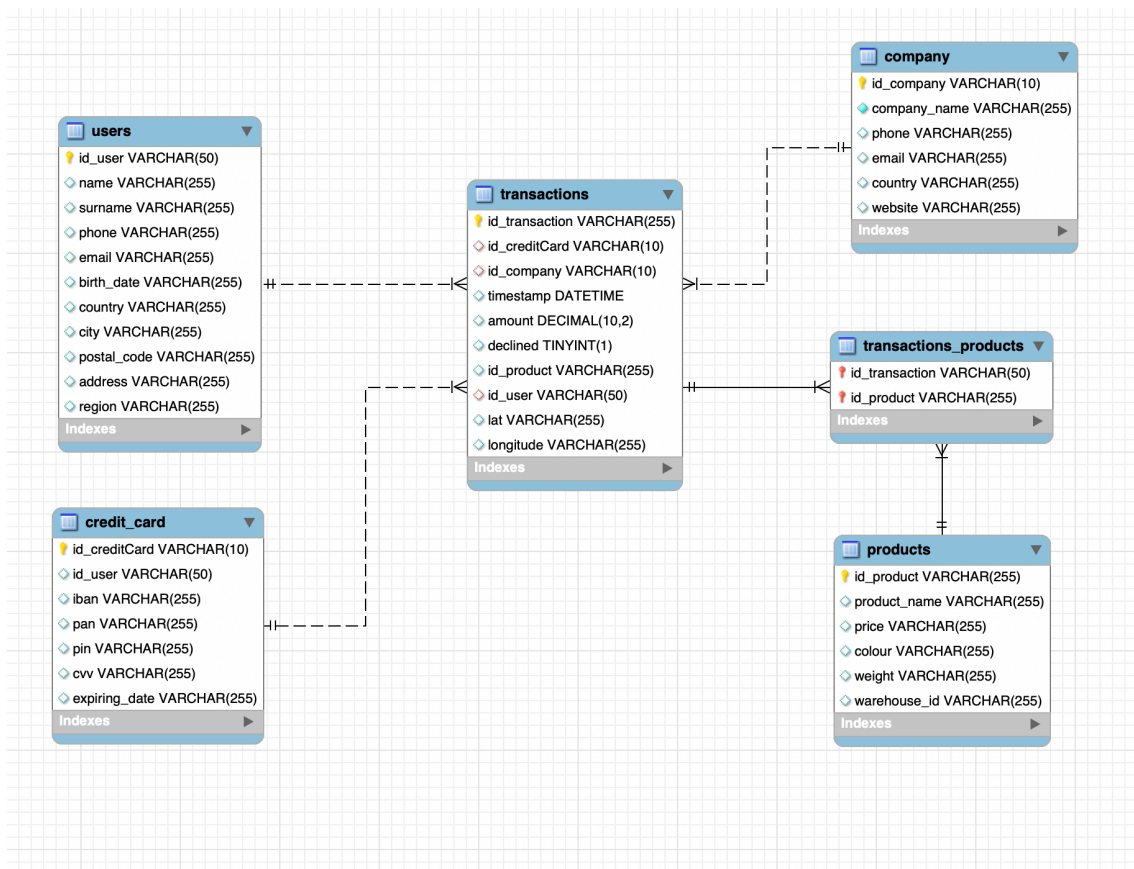
98 -- Se crean las correspondientes FK para que el modelo quede cohesionado
99 • ALTER TABLE transactions_products
100     ADD CONSTRAINT fk_tp_transaction
101     FOREIGN KEY (id_transaction) REFERENCES transactions(id_transaction);
102
103 • ALTER TABLE transactions_products
104     ADD CONSTRAINT fk_tp_product
105     FOREIGN KEY (id_product) REFERENCES products(id_product);
106
107
100% 58:105

```

Action Output

	Time	Action	Response	Duration / Fetch Time
51	12:40:33	SELECT * FROM card_status LIMIT 0, 50000	5000 row(s) returned	0.0037 sec / 0.0052...
52	12:56:21	SELECT product_name AS product, COUNT...	100 row(s) returned	0.375 sec / 0.000059...
53	13:00:28	ALTER TABLE transactions_products ADD CON...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	3.093 sec
54	13:00:37	ALTER TABLE transactions_products ADD CON...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	10.658 sec

Finalmente, el modelo quedaría así, con la tabla intermedia:



Nota: el modelo mantiene una estructura tipo estrella, pero se añade una tabla intermedia para gestionar la relación muchos a muchos entre transacciones y productos. Esto permite separar correctamente los productos asociados a cada transacción y facilitar su análisis.

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```

93 -- Exercici 1
94 -- Necessitem conèixer el nombre de vegades que s'ha venut cada producte.
95
96 • SELECT
97     product_name AS producto,
98     COUNT(*) AS cantidad_ventas
99 FROM products AS p
100 JOIN transactions_products AS tp
101     ON p.id_product = tp.id_product
102 GROUP BY p.id_product
103 ORDER BY cantidad_ventas DESC;
104

```

100%	31:103	Result Grid	Filter Rows: Search	Export:		
producto	cantidad_ventas					
riverlands the duel	2654					
Tully maester Tarly	2635					
duel Direwolf	2609					
the duel warden	2608					
mustafar jinn	2601					
sith Jade	2598					
rock Renlv in	2587					

Result 1
Read Only

	Time	Action	Response	Duration / Fetch Time
1	11:32:30	SELECT product_name AS producto, COUNT(*) AS cantidad_ventas FROM products AS p J...	Error Code: 1046. No database selected Select the de...	0.00074 sec
2	11:32:34	USE world_transactions	0 row(s) affected	0.00043 sec
3	11:32:38	SELECT product_name AS producto, COUNT(*) AS cantidad_ventas FROM products AS p J...	100 row(s) returned	0.409 sec / 0.00006...